

使用 Cloud Workstations 和 Cloud Code 進行開發

來源：<https://codelabs.developers.google.com/innerloop-dev-cloud-workstations-java-springboot?hl=zh-tw>

步驟數：6

瞭解針對在容器化環境中使用 Cloud Workstations 簡化 Java 軟體工程師開發工作流程的功能與功能。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 建立新的 Java 啟動應用程式](#)
4. [4. 逐步完成開發程序](#)
5. [5. 開發簡易的 CRUD REST 服務](#)
6. [6. 恭喜](#)

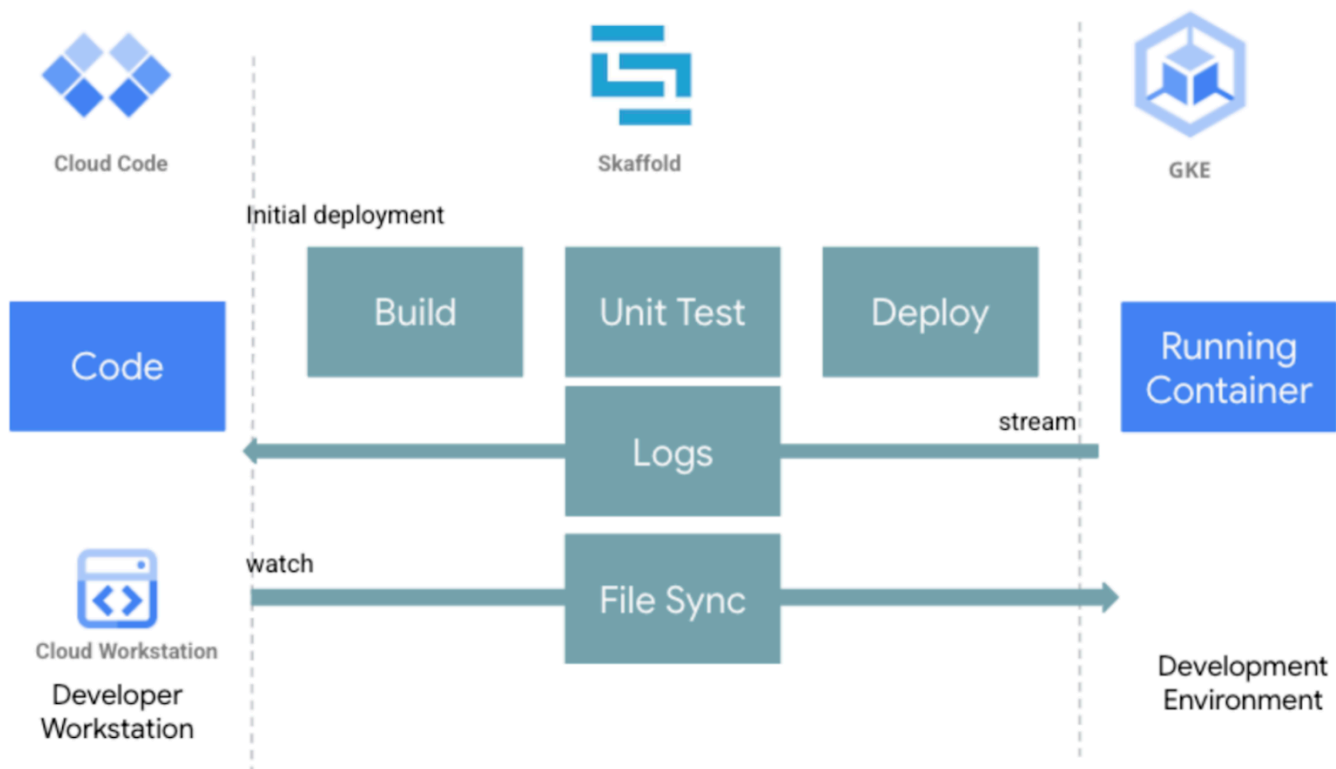
1. 總覽

本實驗室將展示各項功能，協助軟體工程師在容器化環境中開發 Java 應用程式時，簡化開發工作流程。一般容器開發作業需要使用者瞭解容器的詳細資料和容器建構程序。此外，開發人員通常必須中斷流程，離開 IDE 在遠端環境中測試及偵錯應用程式。有了本教學課程中提及的工具和技術，開發人員就能在 IDE 中有效處理容器化應用程式。

學習目標

在本實驗室中，您將瞭解如何在 GCP 中使用容器進行開發，包括：

- 使用 Cloud Workstations 進行 InnerLoop 開發
- 建立新的 Java 啟動應用程式
- 逐步完成開發程序
- 開發簡易的 CRUD REST 服務
- 對 GKE 叢集上的應用程式進行偵錯
- 將應用程式連線至 Cloud SQL 資料庫



2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section is below that, featuring a 'Project name' field with the text 'my-kubernetes-codelab' entered, which is also circled in blue. Below the name field, the 'Project ID' is displayed as 'my-kubernetes-codelab' with a note that it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom of the 'New Project' section are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

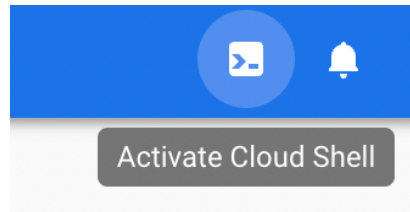
2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell 編輯器

本實驗室專為 Google Cloud Shell 編輯器設計及測試，如要存取編輯器，請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com> 存取 Google 專案。
2. 按一下右上角的 Cloud Shell 編輯器圖示



3. 視窗底部會開啟新窗格
4. 按一下「開啟編輯器」按鈕



5. 編輯器會開啟，右側顯示檔案總管，中央區域則顯示編輯器
6. 畫面底部也應顯示終端機窗格
7. 如果終端機尚未開啟，請使用 `ctrl+`` 鍵組合開啟新的終端機視窗

設定 gcloud

在 Cloud Shell 中，設定專案 ID 和要部署應用程式的區域。分別儲存為 `PROJECT_ID` 和 `REGION` 變數。

```
export REGION=us-central1
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

複製原始碼

本實驗室的原始碼位於 GitHub 的 GoogleCloudPlatform 容器開發人員研討會。使用下列指令複製，然後切換至該目錄。

```
git clone https://github.com/GoogleCloudPlatform/container-developer-workshop.git
cd container-developer-workshop/labs/spring-boot
```

佈建本實驗室使用的基礎架構

在本實驗室中，您將程式碼部署至 GKE，並存取儲存在 Cloud SQL 資料庫中的資料。下方的設定指令碼會為您準備好基礎架構。佈建程序需要 25 分鐘以上。請等待指令碼完成，再前往下一個章節。

```
./setup_with_cw.sh &
```

Cloud Workstations 叢集

在 Cloud 控制台中開啟 [Cloud Workstations](#)。等待叢集處於 **READY** 狀態。

Google Cloud

qwiklabs-gcp-03-520914f07b47

Search Products, resources, docs (/)

Cloud Workstations

Clusters **PREVIEW** [+ CREATE](#) [REFRESH](#)

My resources

My Workstations

Project resources

Workstations

Configurations

Clusters

Clusters manage Workstations during their lifecycle and provide a configurable gateway for network connectivity. Cluster creation can take up to 20 minutes, but is only needed once within a region. [Learn more](#)

Filter Enter property name or value

☐	☐	Status	Name ↑	Location	Network
☐	☒	READY	my-cluster	us-central1	projects/qwiklabs-gcp-03-520914f07b47/global/networks/default

建立工作站設定

如果 Cloud Shell 工作階段已中斷連線，請按一下「重新連線」，然後執行 `gcloud` 指令列指令來設定專案 ID。執行指令前，請將下方的範例專案 ID 換成您的 Qwiklabs 專案 ID。

```
gcloud config set project qwiklabs-gcp-project-id
```

在終端機中執行下列指令碼，建立 Cloud Workstations 設定。

```
cd ~/container-developer-workshop/labs/spring-boot
./workstation_config_setup.sh
```

在「設定」部分下方驗證結果。轉換為「READY」狀態需要 2 分鐘。

Google Cloud

qwiklabs-gcp-03-520914f07b47

Search Products, resources, docs (/)

Cloud Workstations

Configurations **PREVIEW** [+ CREATE](#) [REFRESH](#)

My resources

My Workstations

Project resources

Workstations

Configurations

Clusters

Configurations define settings for Cloud Workstations. Workstations must be associated with a specific configuration at creation time. After you create a configuration, you can create Cloud Workstations from the workstations view. Add creators to a configuration to allow them to create their own workstations. [Learn more](#)

Filter Enter property name or value

☐	☐	Status	Name ↑		Location	Cluster	Machine type
☐	☒	READY	codeoss-java	ADD USERS	us-central1	my-cluster	e2-standard-4

在控制台中開啟 [Cloud Workstations](#)，然後建立新執行個體。

← → ↻ console.cloud.google.com/workstations/list?project=qwiklabs-gcp-01-799f5ac5dfc5

☰ Google Cloud qwiklabs-gcp-01-799f5ac5dfc5 🔍 Search workstation

☞ Cloud Workstations Workstations PREVIEW + CREATE ↻ REFRESH

My resources ^

- My Workstations

Project resources ^

- Workstations
- Configurations
- Clusters

將名稱變更為 `my-workstation`，然後選取現有設定：`codeoss-java`。

☰ Google Cloud qwiklabs-gcp-01-799f5ac5dfc5 🔍 Search Products, resources, docs (/)

☞ Cloud Workstations ← Create workstation PREVIEW

My resources ^

- My Workstations

Project resources ^

- Workstations
- Configurations
- Clusters

<I

Name *
my-workstation

Configuration *
codeoss-java (e2-standard-8) ▼

CREATE CANCEL

驗證「工作站」部分下的結果。

Google Cloud

qwiklabs-gcp-03-520914f07b47

Search Products, resources, docs (/)

Cloud Workstations

Workstations PREVIEW CREATE REFRESH

My resources

My Workstations

Project resources

Workstations

Configurations

Clusters

Cloud Workstations provides managed, on-demand, development environments in the cloud. They can be accessed through a browser UI, a terminal/SSH, or from your local IDE through an SSH bridge. [Learn more](#)

Filter Enter property name or value

Status	Name	Location	Cluster	Configuration	Machine type
Stopped	my-workstation	us-central1	my-cluster	codeoss-java	e2-standard-4

啟動工作站

啟動工作站。

Google Cloud

qwiklabs-gcp-01-036fe18401d1

Search Products, resources, docs (/)

Cloud Workstations

Workstations PREVIEW CREATE REFRESH

My resources

My Workstations

Project resources

Workstations

Configurations

Clusters

Cloud Workstations provides managed, on-demand, development environments in the cloud. They can be accessed through a browser UI, a terminal/SSH, or from your local IDE through an SSH bridge. [Learn more](#)

Filter Enter property name or value

Status	Name	Location	Cluster	Configuration
Running	my-workstation	us-central1	my-cluster	codeoss-java

按一下網址列中的圖示，允許第三方 Cookie。



Workstations – Cloud Worksta

Get Started — Code OSS for C

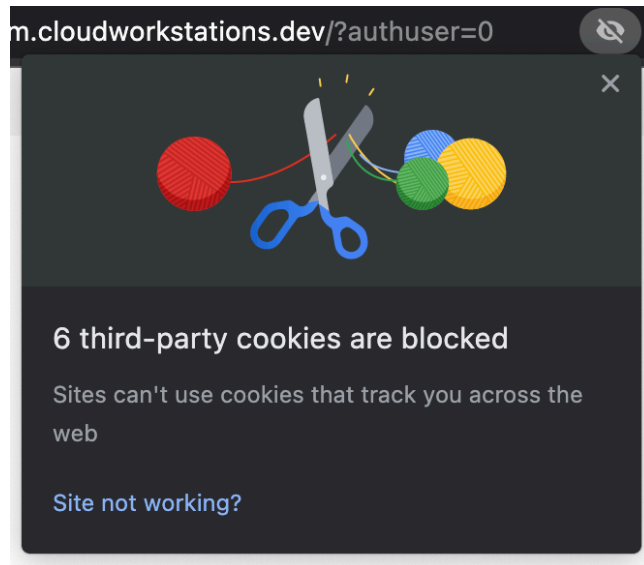
80-my-workstation.cluster-ixavuiofpdfsv224plj24ejzm.cloudworkstations.dev/?authuser=0

Incognito

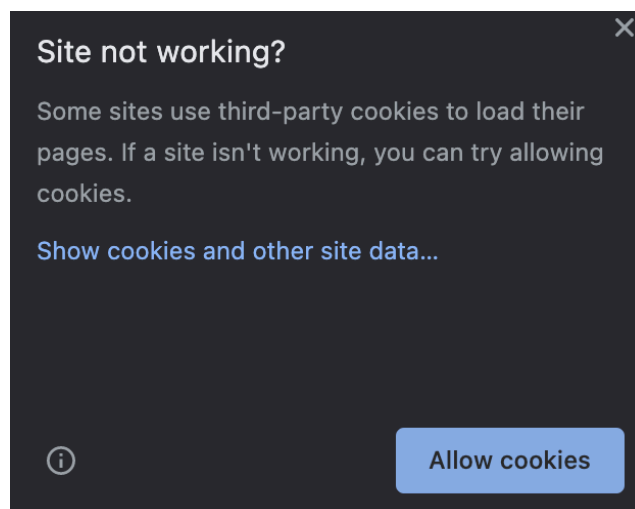
Third-party cookie blocking

Get Started

按一下「網站無法正常運作嗎？」。



按一下「允許 Cookie」。



工作站啟動後，您會看到 Code OSS IDE。在工作站 IDE 的「開始使用」頁面中，按一下「標示為完成」



Get Started with Code OSS in the Web

Discover the best customizations to make Code OSS in the Web yours.

☐ **Choose the look you want**

The right color palette helps you focus on your code, is easy on your eyes, and is simply more fun to use.

[Browse Color Themes](#)

Tip: Use keyboard shortcut (**⌘K ⌘T**)

☐ **One shortcut to access everything**

☐ **Just the right amount of UI**

☐ **Rich support for all your languages**

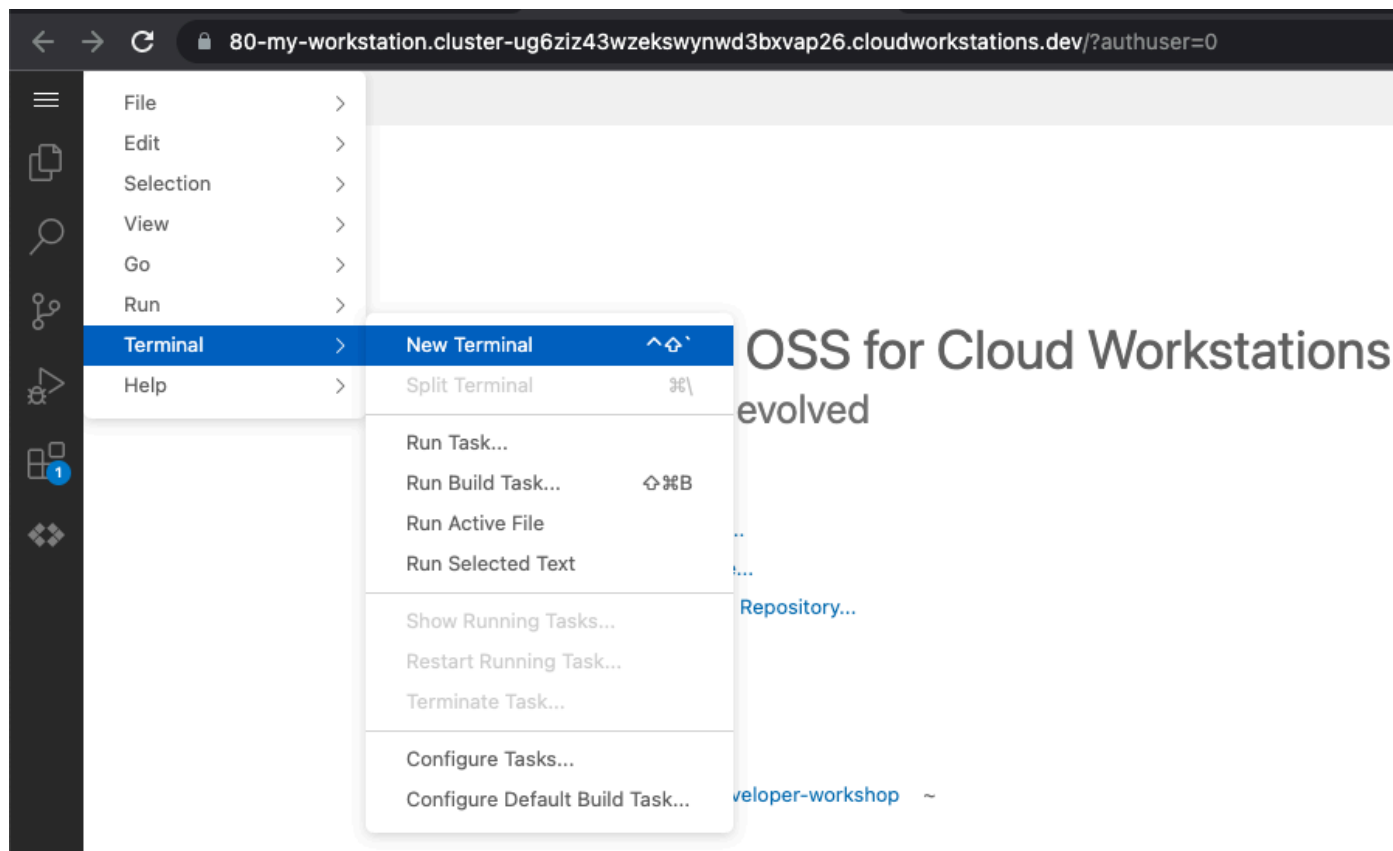
☒ **Mark Done**

[Next Section →](#)

步驟 3 · 約 0 分鐘

3. 建立新的 Java 啟動應用程式

在本節中，您將從頭開始建立新的 Java Spring Boot 應用程式，並使用 spring.io 提供的範例應用程式。開啟新的終端機。

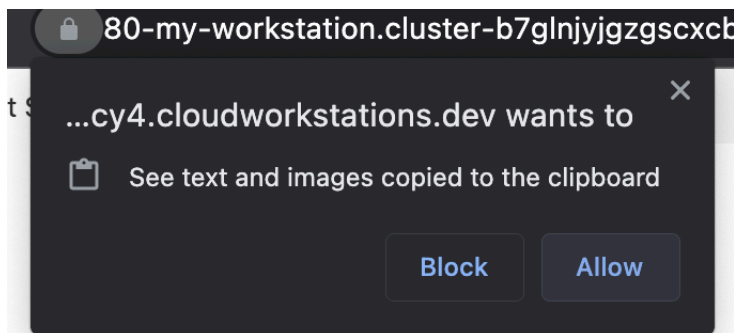


複製範例應用程式

1. 建立入門應用程式

```
curl https://start.spring.io/starter.zip -d dependencies=web -d type=maven-project -d javaVersion=17 -d packageName=com.example.springboot -o sample-app.zip
```

如果看到這則訊息，請按一下「允許」按鈕，這樣就能複製並貼到工作站。



2. 解壓縮應用程式

```
unzip sample-app.zip -d sample-app
```

3. 開啟「sample-app」資料夾

```
cd sample-app && code-oss-cloud-workstations -r --folder-uri="$PWD"
```

新增 spring-boot-devtools 和 Jib

如要啟用 Spring Boot DevTools，請在編輯器的檔案總管中尋找並開啟 `pom.xml`。接著，在說明行 (讀取 `<description>Demo project for Spring Boot</description>`) 後方貼上下列程式碼

1. 在 `pom.xml` 中新增 `spring-boot-devtools`

開啟專案根目錄中的 `pom.xml`。在 `Description` 項目後方新增下列設定。

`pom.xml`

```
<!-- Spring profiles-->
<profiles>
  <profile>
    <id>sync</id>
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
      </dependency>
    </dependencies>
  </profile>
</profiles>
```

2. 在 `pom.xml` 中啟用 `jib-maven-plugin`

Jib 是 Google 推出的開放原始碼 Java 容器化工具，可讓 Java 開發人員使用熟悉的 Java 工具建構容器。Jib 是一種快速又簡單的容器映像檔建構工具，可處理將應用程式封裝為容器映像檔的所有步驟。您不需要編寫 `Dockerfile` 或安裝 Docker，而且這項功能直接整合至 Maven 和 Gradle。

在 `pom.xml` 檔案中向下捲動，並更新 `Build` 區段，加入 Jib 外掛程式。完成後，建構部分應如下所示。

`pom.xml`

```

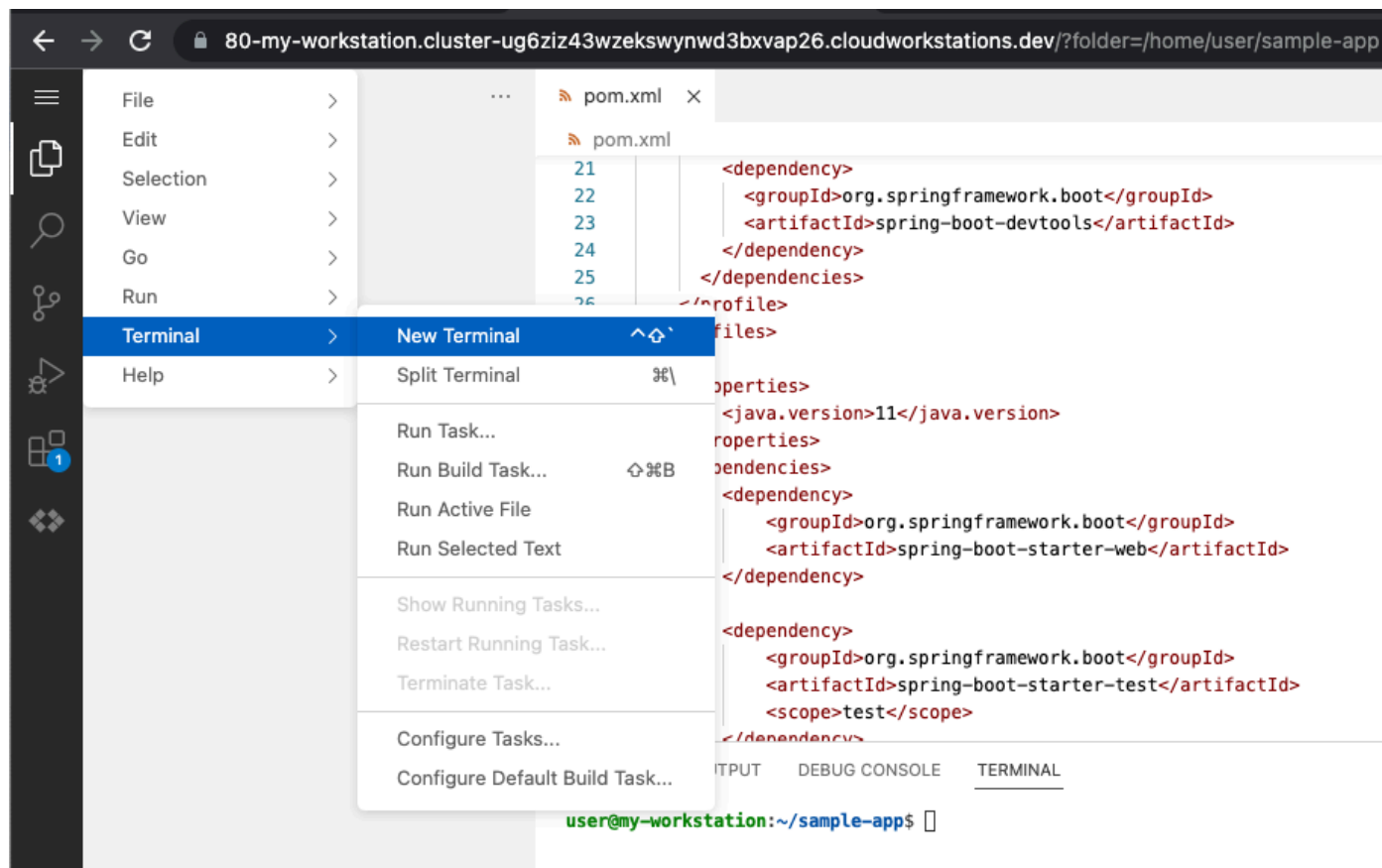
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <!-- Jib Plugin-->
    <plugin>
      <groupId>com.google.cloud.tools</groupId>
      <artifactId>jib-maven-plugin</artifactId>
      <version>3.2.0</version>
    </plugin>
    <!-- Maven Resources Plugin-->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-resources-plugin</artifactId>
      <version>3.1.0</version>
    </plugin>
  </plugins>
</build>

```

產生資訊清單

[Scaffold](#) 提供整合式工具，可簡化容器開發作業。在這個步驟中，您將初始化 Scaffold，系統會自動建立基本 Kubernetes YAML 檔案。這個程序會嘗試找出含有容器映像檔定義的目錄 (例如 Dockerfile)，然後為每個目錄建立部署和服務資訊清單。

在終端機中執行下列指令，即可開始這項程序。



1. 在終端機中執行下列指令

```
scaffold init --generate-manifests
```

2. 出現提示時：

- 使用箭頭將游標移至 **Jib Maven Plugin**
- 按下空格鍵選取選項。
- 按下 Enter 鍵即可繼續

3. 輸入 **8080** 做為通訊埠

4. 輸入 **y** 儲存設定

兩個檔案會新增至工作區 `scaffold.yaml` 和 `deployment.yaml`

Scaffold 輸出內容：

```
Generated manifest deployment.yaml was written
Configuration scaffold.yaml was written
You can now run [scaffold build] to build the artifacts
or [scaffold run] to build and deploy
or [scaffold dev] to enter development mode, with auto-redeploy
user@my-workstation:~/sample-app$
```

更新應用程式名稱

設定中包含的預設值目前與應用程式名稱不符。更新檔案，參照應用程式名稱，而非預設值。

1. 變更 Scaffold 設定中的項目

- 開啟「`scaffold.yaml`」
- 選取目前設為 `pom-xml-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `demo-app`

2. 變更 Kubernetes 設定中的項目

- 開啟 `deployment.yaml` 檔案
- 選取目前設為 `pom-xml-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `demo-app`

啟用自動同步模式

為獲得最佳化的熱重載體驗，您將使用 Jib 提供的[同步功能](#)。在本步驟中，您將設定 Scaffold，在建構程序中使用這項功能。

請注意，您在 Scaffold 設定中設定的「sync」設定檔，會運用您在上一步設定的 Spring「sync」設定檔，其中已啟用對 `spring-dev-tools` 的支援。

1. 更新 Scaffold 設定

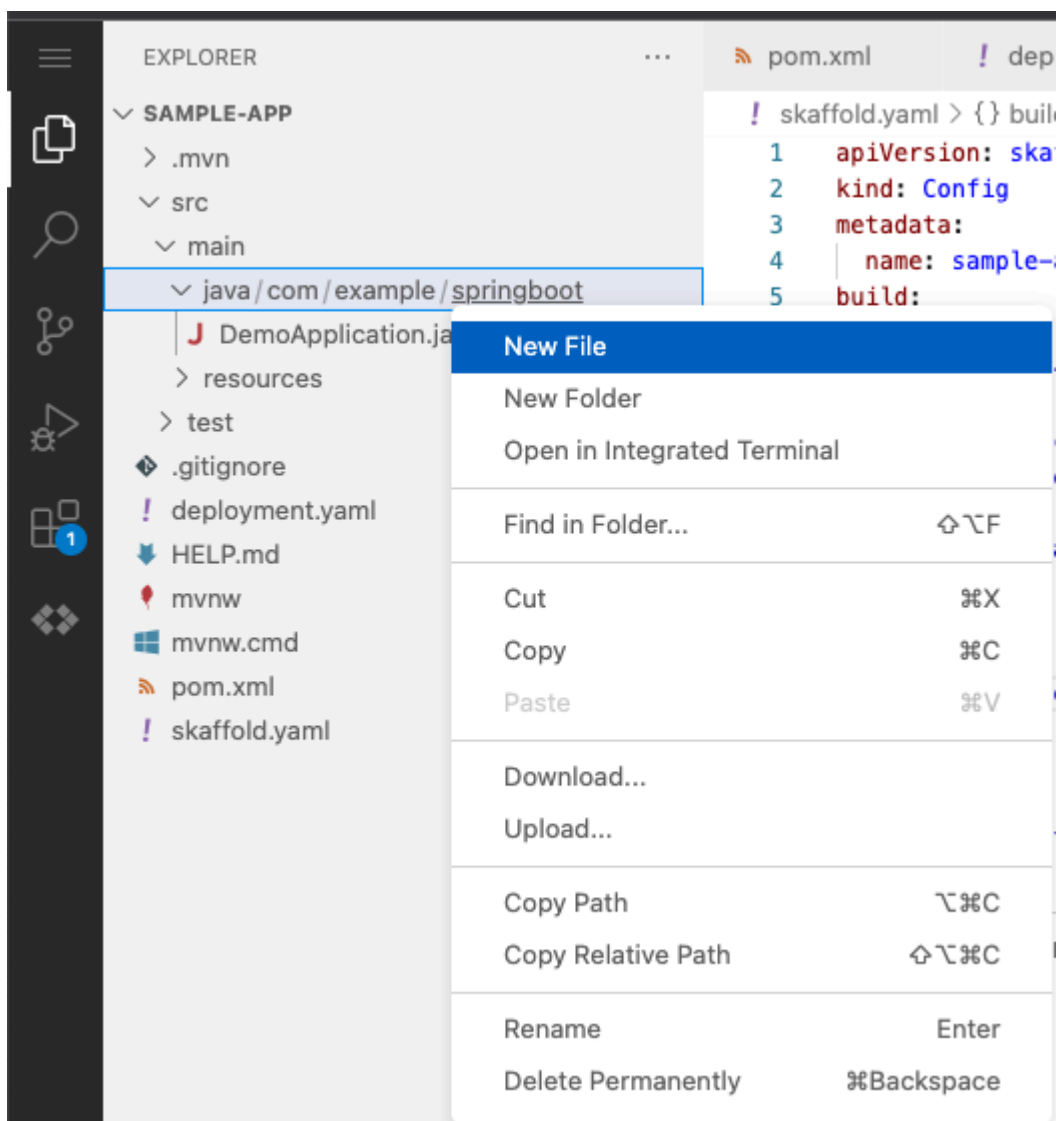
在 `scaffold.yaml` 檔案中，將檔案的整個建構部分替換為下列規格。請勿變更檔案的其他部分。

skaffold.yaml

```
build:
  artifacts:
    - image: demo-app
  jib:
    project: com.example:demo
    type: maven
    args:
      - --no-transfer-progress
      - -Psync
    fromImage: gcr.io/distroless/java17-debian11:debug
  sync:
    auto: true
```

新增預設路由

在 `/src/main/java/com/example/springboot/` 資料夾中建立名為 `HelloController.java` 的檔案。



將下列內容貼到檔案中，建立預設的 HTTP 路由。

HelloController.java

```
package com.example.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.beans.factory.annotation.Value;

@RestController
public class HelloController {

    @Value("${target:local}")
    String target;

    @GetMapping("/")
    public String hello()
    {
        return String.format("Hello from your %s environment!", target);
    }
}
```

步驟 4 · 約 0 分鐘

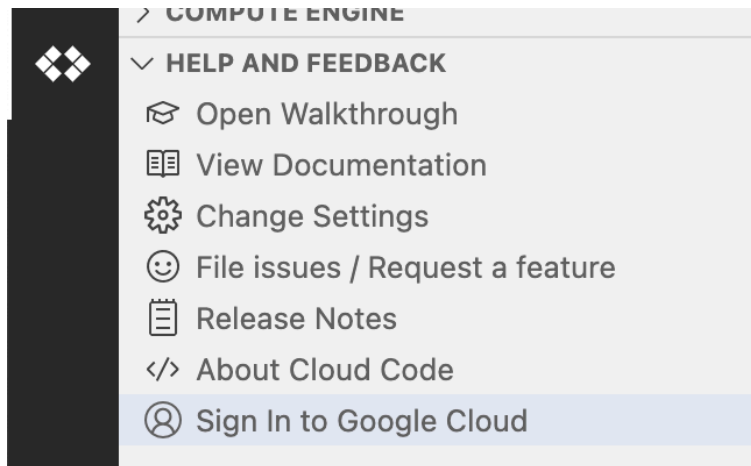
4. 逐步完成開發程序

在本節中，您將使用 Cloud Code 外掛程式逐步完成幾個步驟，瞭解基本程序，並驗證入門應用程式的設定。

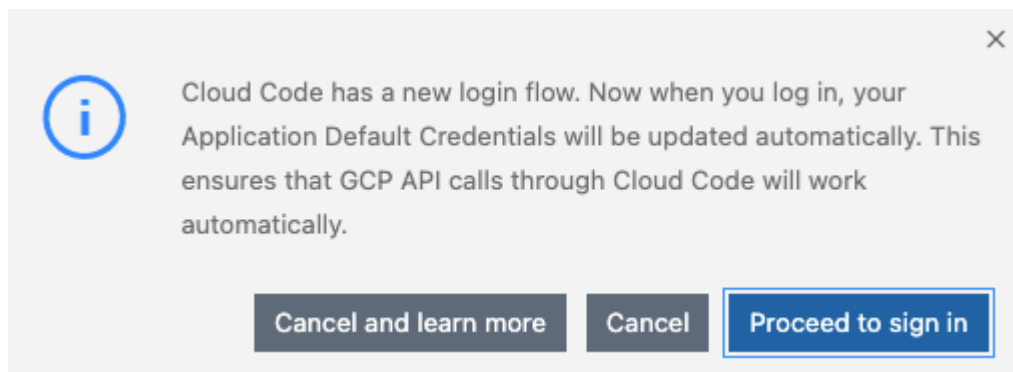
Cloud Code 整合了 Skaffold，可簡化開發程序。在後續步驟中部署至 GKE 時，Cloud Code 和 Skaffold 會自動建構容器映像檔、將其推送至 Container Registry，然後將應用程式部署至 GKE。這項作業會在幕後進行，將詳細資料從開發人員流程中抽象化。此外，Cloud Code 也為以容器為基礎的開發作業提供傳統的偵錯和熱同步功能，進一步提升開發流程。

登入 Google Cloud

按一下 Cloud Code 圖示，然後選取「Sign in to Google Cloud」(登入 Google Cloud)：



按一下「Proceed to sign in」(繼續登入)。



查看終端機的輸出內容並開啟連結：

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL

```
/usr/bin/gcloud auth login --update-adc --add-quota-project-to-adc --quiet --verbosity warning
Go to the following link in your browser:
```

```
https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=https%3A%2F%2Fsdk.cloud.google.com%2Fauth
code.html&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platform+https%3A%2F%2Fwww.googleapis.co
m%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com
%2Fauth%2Faccounts.reauth&state=tSdQJcJe4FdZ4Ts1cqj5aCHHisjjB76prompt=consent&access_type=offline&code_challenge=hQXEautMt0jaZWrILi2xNAhh22JswrDB8ZCoCIYg0aY&code_challe
nge_method=S256
```

```
Enter authorization code: █
```

使用 Qwiklabs 學生憑證登入。

 Sign in with Google



Choose an account

to continue to **Google Cloud SDK**

 student-00-540866a50ab4@qwiklabs.net

 **Use another account**

To continue, Google will share your name, email address, language preference, and profile picture with Google Cloud SDK.

選取「允許」：

 Sign in with Google



Google Cloud SDK wants to access your Google Account

 student-00-540866a50ab4@qwiklabs.net

This will allow **Google Cloud SDK** to:

- See, edit, configure, and delete your Google Cloud data and see the email address for your Google Account. 
- View and sign in to your Google Cloud SQL instances 
- View and manage your Google Compute Engine resources 
- View and manage your applications deployed on Google App Engine 

Make sure you trust Google Cloud SDK

You may be sharing sensitive info with this site or app. You can always see or remove access in your [Google Account](#).

Learn how Google helps you [share data safely](#).

See Google Cloud SDK's Privacy Policy and Terms of Service.

Cancel

Allow

複製驗證碼，然後返回「工作站」分頁。



Sign in to the gcloud CLI

You are seeing this page because you ran the following command in the gcloud CLI from this or another machine. If this is not the case, close this tab.

```
gcloud auth login --no-launch-browser
```

Enter the following verification code in gcloud CLI on the machine you want to log into. This is a credential similar to your password and should not be shared with others.

```
4/0AfgeXvs8ORnkDVPb33lRxLZdqK7X8n6yAo
EV53FBnUCkmTQuUaGEZXo0BWl4Z7U07wbyXg
```

Copy

You can close this tab when you're done.

貼上驗證碼，然後按下 Enter 鍵。

```
/usr/bin/gcloud auth login --update-adc --add-quota-project-to-adc --quiet --verbosity warning
Go to the following link in your browser:
```

```
https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=https%3A%2F%2Fsdk.cloud.google.com%2Fauth
code.html&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platform+https%3A%2F%2Fwww.googleapis.co
m%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com
%2Fauth%2Faccounts.reauth&state=tSdQJcJe4FdZ4Ts1cqj5aCHHijjB7&prompt=consent&access_type=offline&code_challenge=hQXEautMt0jaZwrIL12xNAhh22JswrDB8ZCoCIYg0aY&code_challe
nge_method=S256
```

```
Enter authorization code: 4/0AfgeXvs8ORnkDVPb33lRxLZdqK7X8n6yAoEV53FBnUCkmTQuUaGEZXo0BWl4Z7U07wbyXg
WARNING: Cannot find a project to insert into application default credentials (ADC) as a quota project.
Run $gcloud auth application-default set-quota-project to insert a quota project to ADC.
```

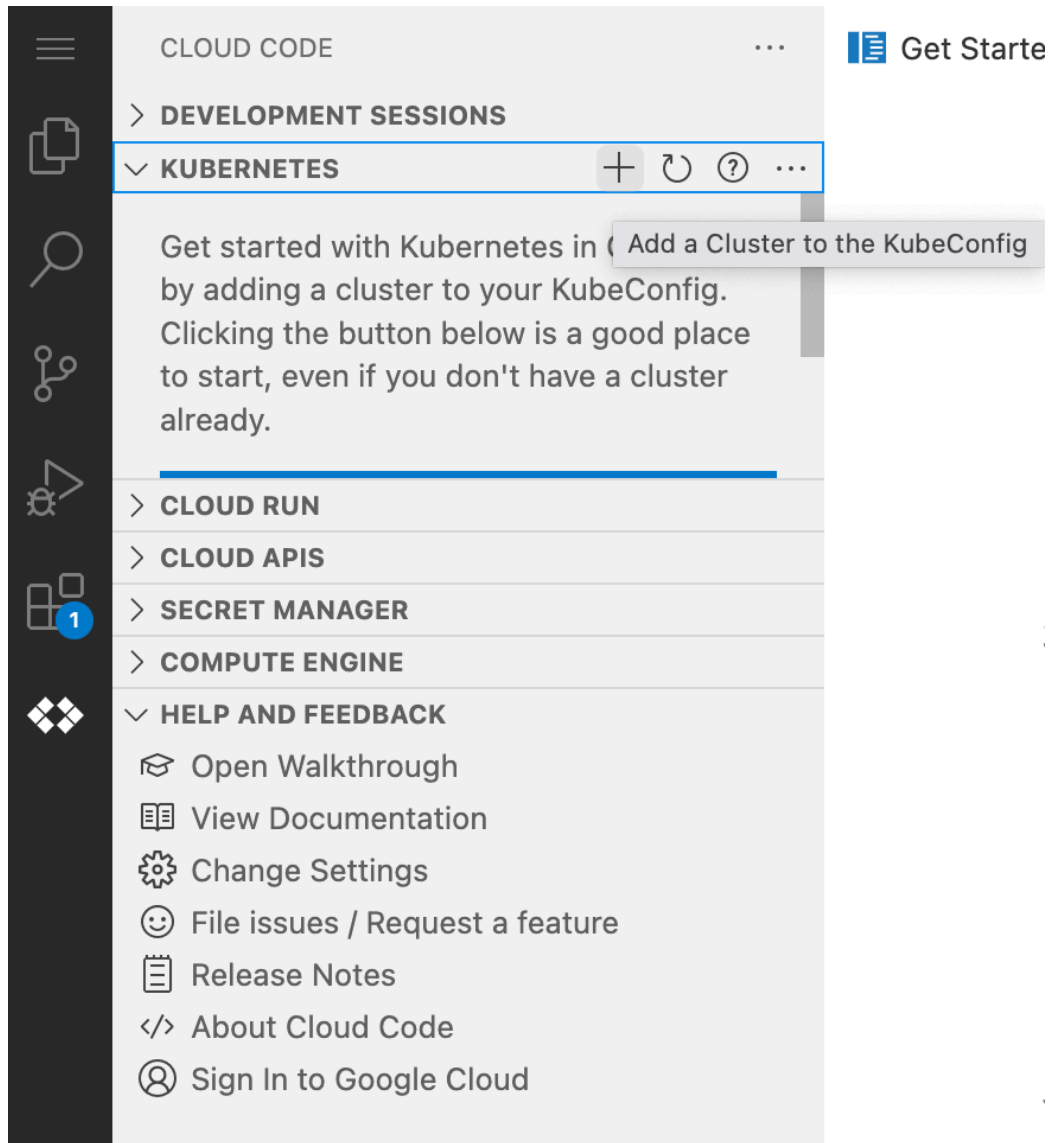
```
Application default credentials (ADC) were updated.
```

```
You are now logged in as [student-00-540866a50ab4@qwiklabs.net].
Your current project is [None]. You can change this setting by running:
$ gcloud config set project PROJECT_ID
```

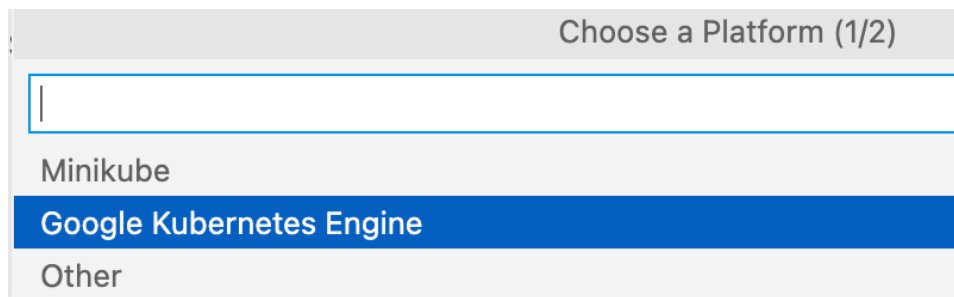
```
Press any key to exit.[]
```

新增 Kubernetes 叢集

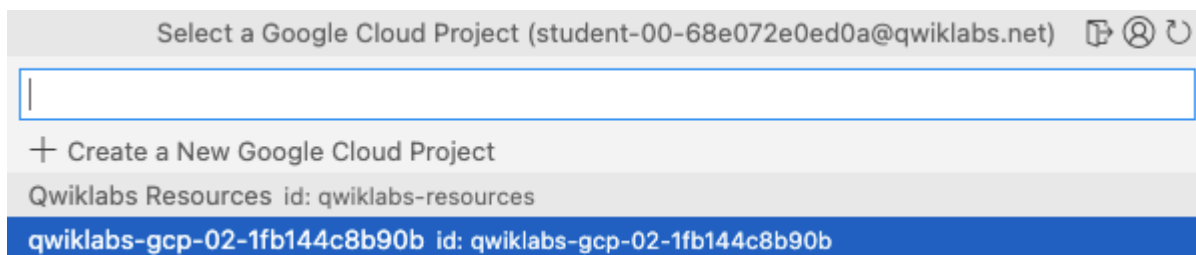
1. 新增叢集



2. 選取 Google Kubernetes Engine :



3. 選取專案。



4. 選取在初始設定中建立的「quote-cluster」。

←

Create or choose a GKE Cluster (project: qwiklabs-gcp-01-799f5ac5dfc5) (2/2)

↺ ↻

+ Create a New GKE Cluster

quote-cluster us-central1 - RUNNING

☰

☰

🔍

🔗

⚙️

📦 1

🎯

CLOUD CODE

> DEVELOPMENT SESSIONS

✓ KUBERNETES

- 📄 Default KubeConfig /home/user/.kube/config
- ✓ gke_qwiklabs-gcp-01-799f5ac5dfc5_us-central1_quote-cluster STANDARD - ACTI... [🔗](#)
 - > GKE Cluster Properties
 - ✓ Contexts
 - * gke_qwiklabs-gcp-01-799f5ac5dfc5_us-central1_quote-cluster Namespace: default, U...
 - ✓ Namespaces

> CLOUD RUN

> CLOUD APIS

> SECRET MANAGER

> COMPUTE ENGINE

✓ HELP AND FEEDBACK

使用 gcloud CLI 設定目前的專案 ID

從 Qwiklabs 頁面複製這個實驗室的專案 ID。

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked.
[Learn more.](#)

Console

Username

student-00-1f28f50e2fc5

📋

Password

CJu118bpNfZE

📋

Project ID

qwiklabs-gcp-04-d75bbac

📋

執行 gcloud CLI 指令，設定專案 ID。請先將範例專案 ID 改為對應的值，再執行指令。

```
gcloud config set project qwiklabs-gcp-project-id
```

輸出內容範例：

```
user@my-workstation:~/sample-app$ gcloud config set project qwiklabs-gcp-03-520914f07b47
Updated property [core/project].
user@my-workstation:~/sample-app$
```

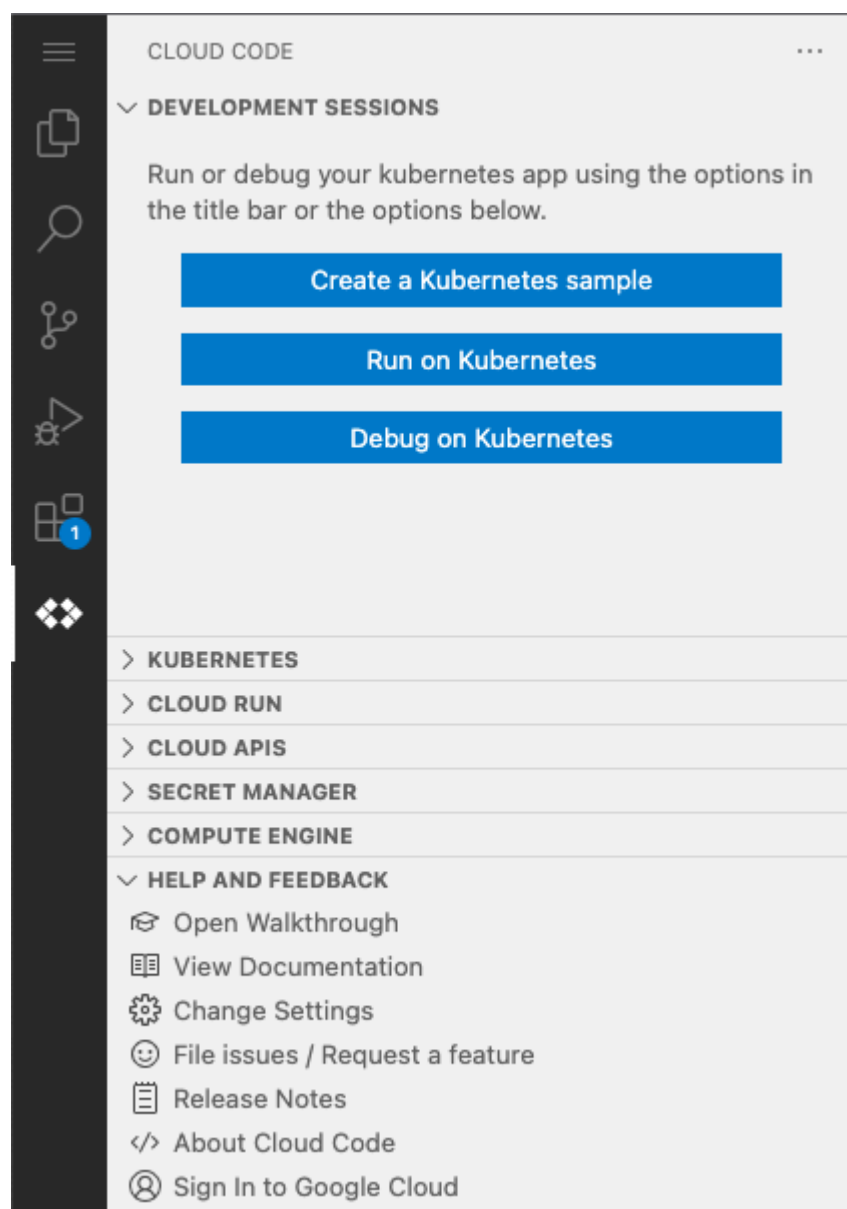
在 Kubernetes 上偵錯

1. 在左側窗格底部選取 Cloud Code。

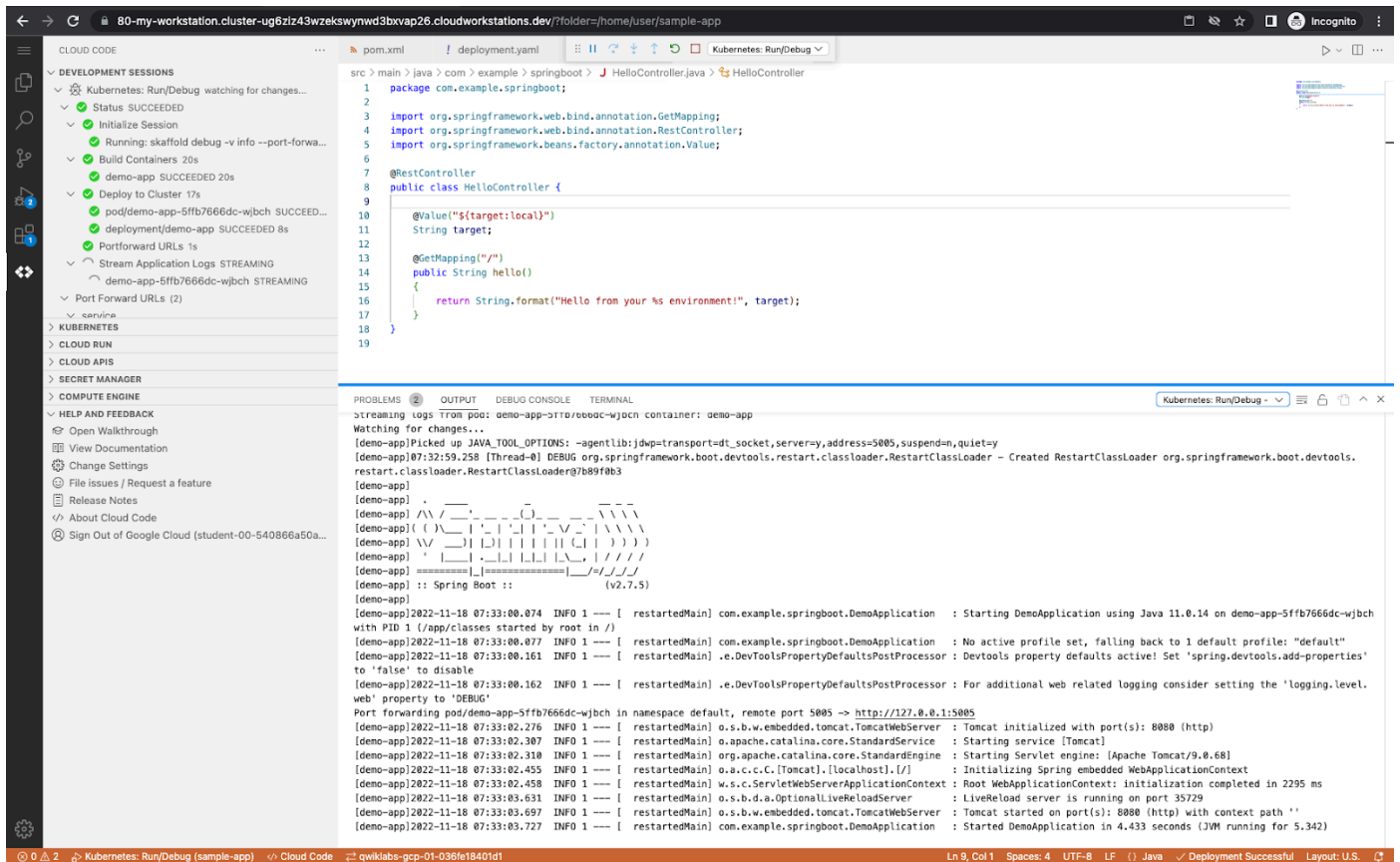


2. 在「DEVELOPMENT SESSIONS」下方顯示的面板中，選取「Debug on Kubernetes」。

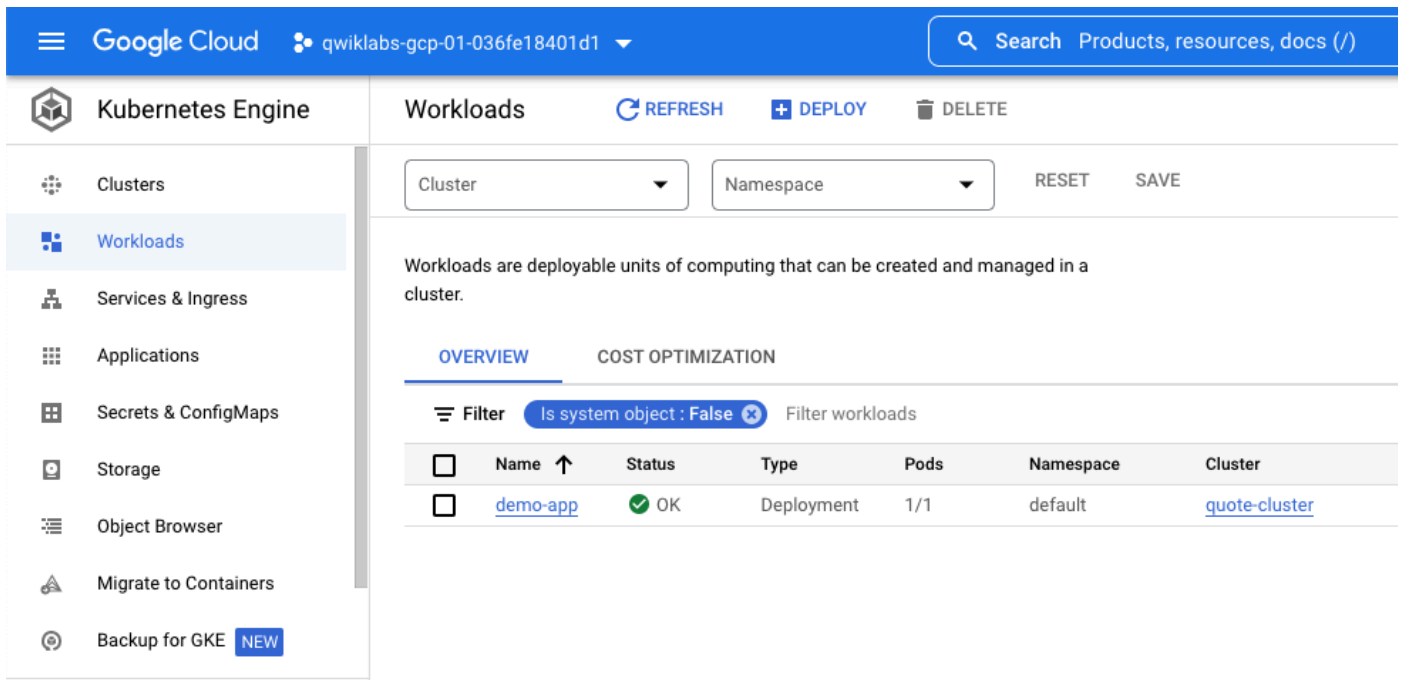
如果沒有看到這個選項，請向下捲動。



3. 選取「是」即可使用目前的結構定義。



8. 在 [Cloud 控制台](#) 中，查看 GKE 上部署的應用程式。



9. 如要返回簡化檢視畫面，請在「OUTPUT」分頁的下拉式選單中選取「Kubernetes: Run/Debug」。

10. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/demo-app status completed successfully`，並列出網址：「Forwarded URL from service demo-app: <http://localhost:8080>」（從服務 demo-app 轉送的網址：<http://localhost:8080>）

11. 在 Cloud Code 終端機中，將游標懸停在輸出內容中的網址 (<http://localhost:8080>)，然後在顯示的工具提示中選取「Follow link」（追蹤連結）。

```
*****URLs*****  
Forwarded URL from service demo-app: http://localhost:8080  
Debuggable container started pod/demo-app-5ffb7666dc-wjbch:demo-app (default)  
Update succeeded  
*****
```

Follow link (cmd + click)

Forwarded URL from service demo-app: <http://localhost:8080>

Debuggable container started pod/demo-app-5ffb7666dc-wjbch:demo-app (default)

Update succeeded

```
*****
```

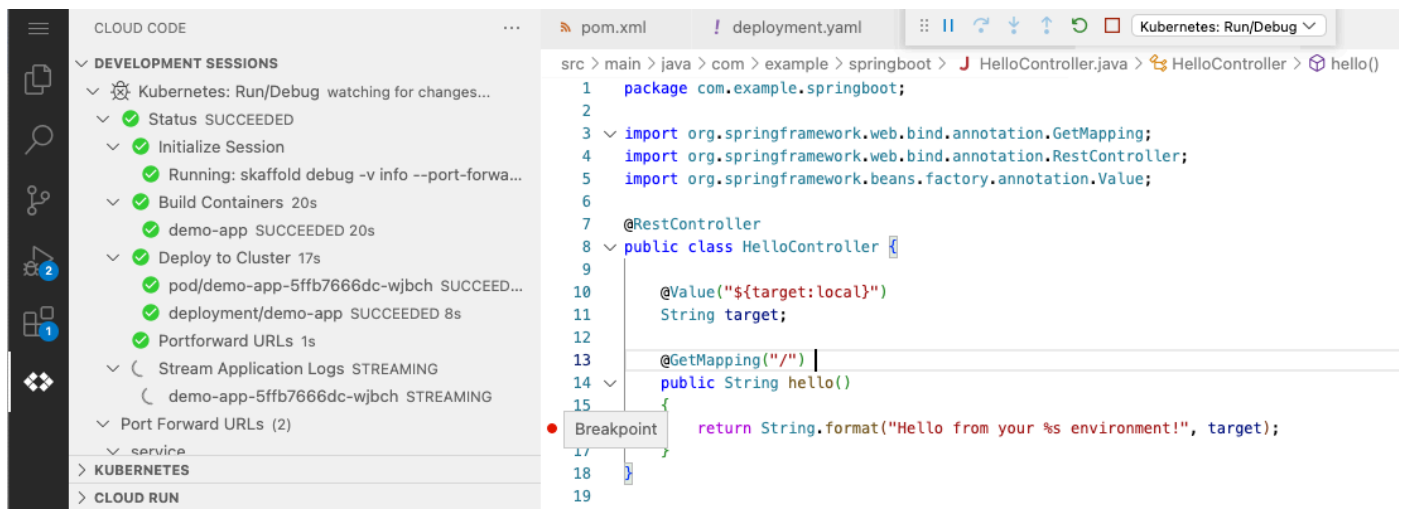
系統會開啟新分頁，並顯示下列輸出內容：

← → ↺ 8080-my-workstation.cluster-ug6ziz43wzekswynwd3bxvap26.cloudworkstations.dev

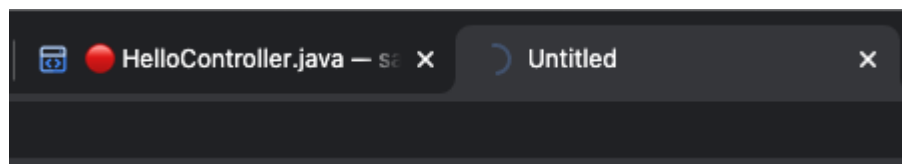
Hello from your local environment!

運用中斷點

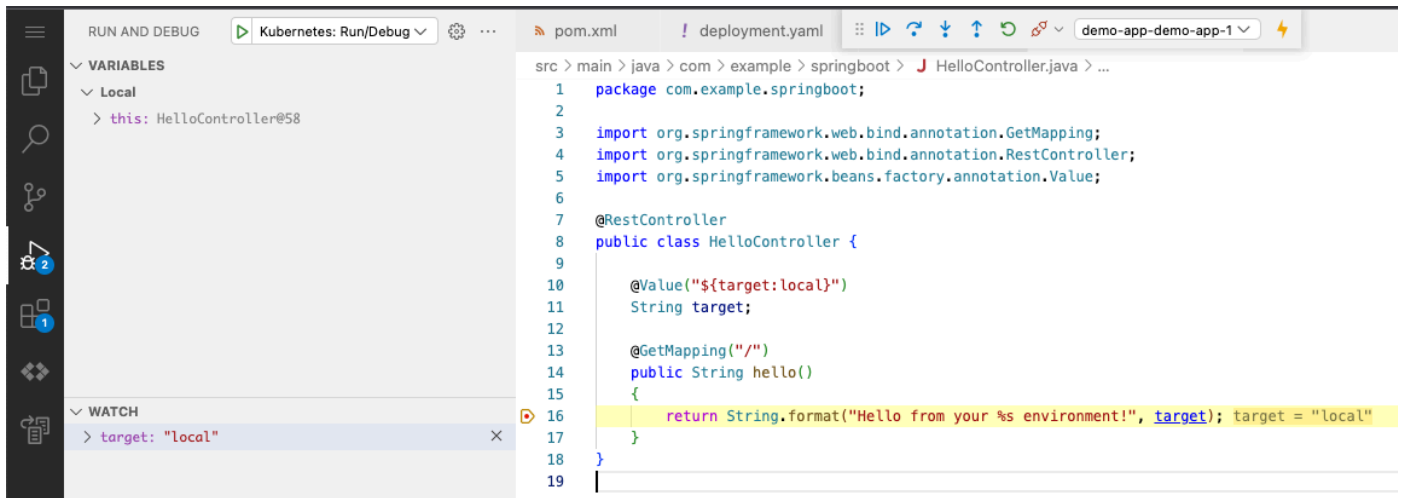
1. 開啟位於 `/src/main/java/com/example/springboot/HelloController.java` 的 `HelloController.java` 應用程式。
2. 找出根路徑的傳回敘述，內容為 `return String.format("Hello from your %s environment!", target);`
3. 點選行號左側的空白處，在該行新增中斷點。系統會顯示紅色指標，表示已設定中斷點



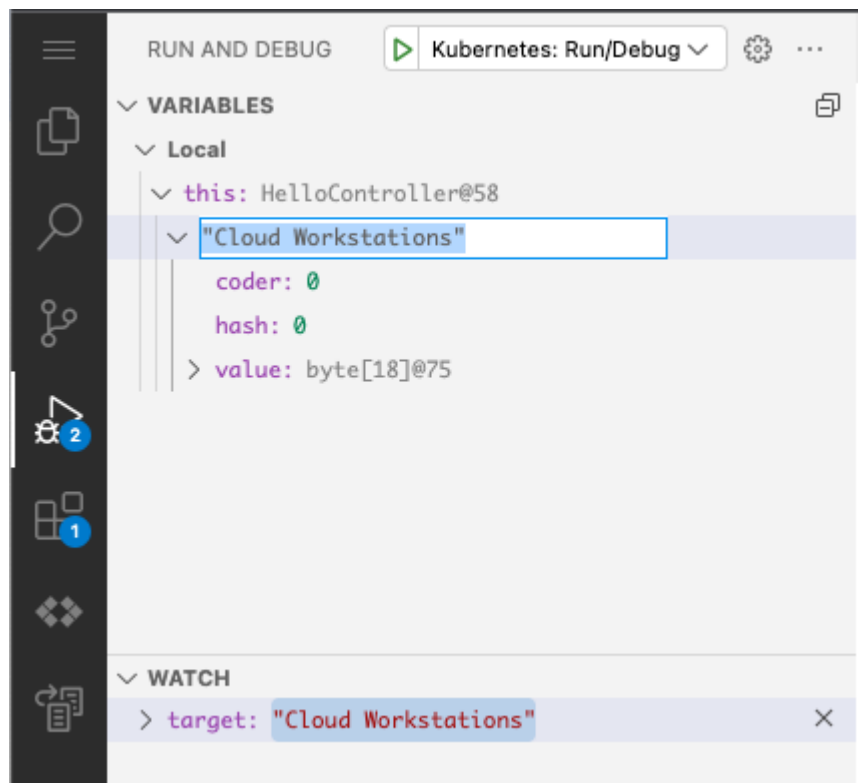
4. 重新載入瀏覽器，請注意偵錯工具會在該中斷點停止程序，並允許您調查在 GKE 中遠端執行的應用程式變數和狀態



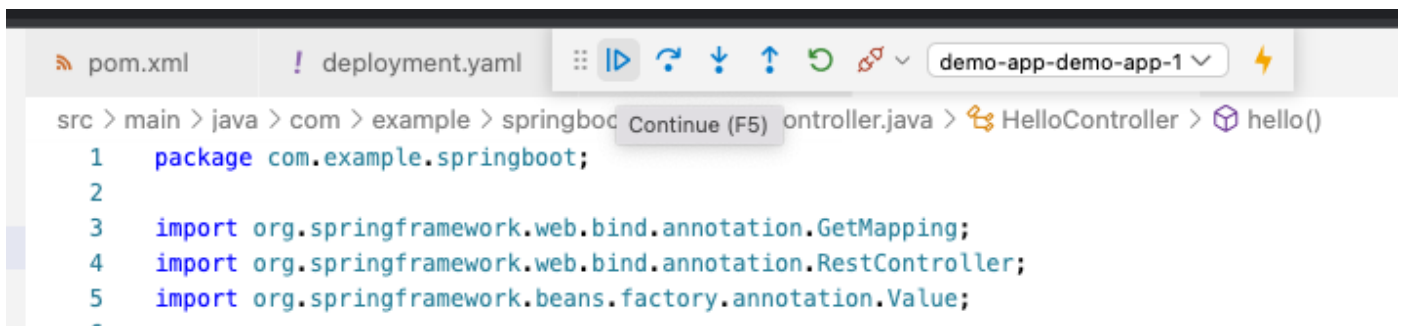
5. 按一下變數部分，直到找到「目標」變數為止。
6. 觀察目前值為「local」



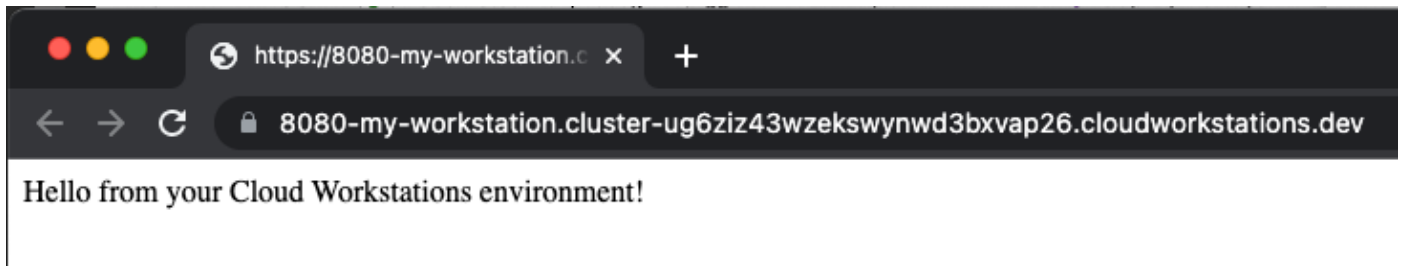
7. 按兩下變數名稱「target」，然後在彈出式視窗中，
將值變更為「Cloud Workstations」



8. 按一下偵錯控制面板中的「繼續」按鈕



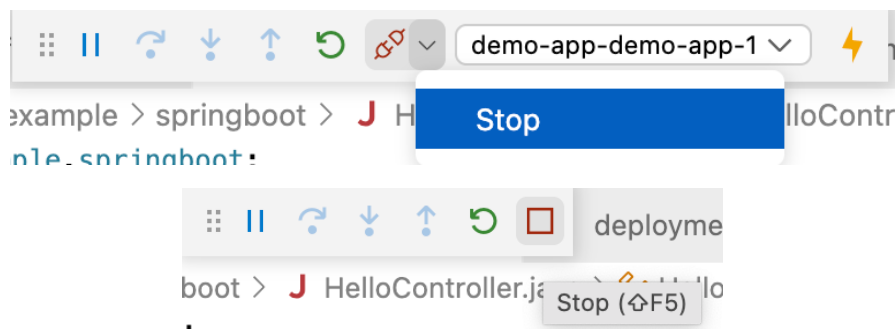
9. 在瀏覽器中查看回應，現在應該會顯示您剛輸入的更新值。



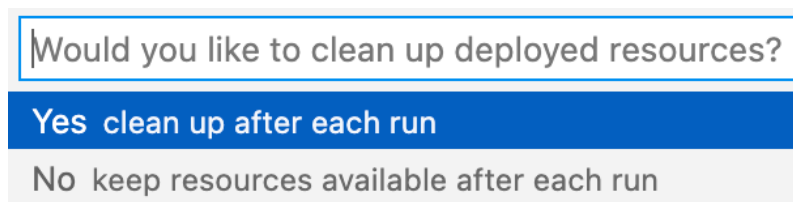
10. 按一下行號左側的紅色指標，即可移除中斷點。這樣一來，您在這個實驗室中繼續操作時，程式碼就不會在這行停止執行。

熱重載

1. 將陳述式變更為傳回其他值，例如「Hello from %s Code」
2. 檔案會自動儲存並同步到 GKE 中的遠端容器
3. 重新整理瀏覽器即可查看更新後的結果。
4. 按一下偵錯工具列中的紅色正方形，停止偵錯工作階段



選取「是，每次執行後都清理」。



5. 開發簡易的 CRUD REST 服務

此時，您的應用程式已完全設定為容器化開發，且您已透過 Cloud Code 逐步瞭解基本開發工作流程。在接下來的章節中，您將練習所學內容，新增連線至 Google Cloud 中代管資料庫的 REST 服務端點。

設定依附元件

應用程式程式碼會使用資料庫保存 REST 服務資料。在 `pom.xml` 中新增下列內容，確保依附元件可用

1. 開啟 `pom.xml` 檔案，並在設定的依附元件區段中新增下列內容

`pom.xml`

```
<!-- Database dependencies-->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>javax.persistence</groupId>
  <artifactId>javax.persistence-api</artifactId>
  <version>2.2</version>
</dependency>
```

編寫 REST 服務程式碼

`Quote.java`

在 `/src/main/java/com/example/springboot/` 中建立名為 `Quote.java` 的檔案，然後複製下列程式碼。這會定義應用程式中使用的 Quote 物件的 Entity 模型。

```
package com.example.springboot;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

import java.util.Objects;

@Entity
@Table(name = "quotes")
public class Quote
{
    @Id
    @Column(name = "id")
    private Integer id;

    @Column(name="quote")
    private String quote;

    @Column(name="author")
    private String author;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getQuote() {
        return quote;
    }

    public void setQuote(String quote) {
        this.quote = quote;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
    }
}
```

```

    if (o == null || getClass() != o.getClass()) {
        return false;
    }
    Quote quote1 = (Quote) o;
    return Objects.equals(id, quote1.id) &&
        Objects.equals(quote, quote1.quote) &&
        Objects.equals(author, quote1.author);
}

@Override
public int hashCode() {
    return Objects.hash(id, quote, author);
}
}

```

QuoteRepository.java

在 `src/main/java/com/example/springboot` 建立名為 `QuoteRepository.java` 的檔案，然後複製下列程式碼

```

package com.example.springboot;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

public interface QuoteRepository extends JpaRepository<Quote,Integer> {

    @Query( nativeQuery = true, value =
        "SELECT id,quote,author FROM quotes ORDER BY RANDOM() LIMIT 1")
    Quote findRandomQuote();
}

```

這段程式碼會使用 JPA 持續保存資料。這個類別會擴充 Spring `JpaRepository` 介面，並允許建立自訂程式碼。在您新增的程式碼中，加入 `findRandomQuote` 自訂方法。

QuoteController.java

如要公開服務的端點，`QuoteController` 類別會提供這項功能。

在 `src/main/java/com/example/springboot` 建立名為 `QuoteController.java` 的檔案，並複製下列內容

```

package com.example.springboot;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class QuoteController {

    private final QuoteRepository quoteRepository;

    public QuoteController(QuoteRepository quoteRepository) {
        this.quoteRepository = quoteRepository;
    }

    @GetMapping("/random-quote")
    public Quote randomQuote()
    {
        return quoteRepository.findRandomQuote();
    }

    @GetMapping("/quotes")
    public ResponseEntity<List<Quote>> allQuotes()
    {
        try {
            List<Quote> quotes = new ArrayList<Quote>();

            quoteRepository.findAll().forEach(quotes::add);

            if (quotes.size()==0 || quotes.isEmpty())
                return new ResponseEntity<List<Quote>>(HttpStatus.NO_CONTENT);

            return new ResponseEntity<List<Quote>>(quotes, HttpStatus.OK);
        } catch (Exception e) {
            System.out.println(e.getMessage());
            return new ResponseEntity<List<Quote>>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }

    @PostMapping("/quotes")
    public ResponseEntity<Quote> createQuote(@RequestBody Quote quote) {

```



```

    try {
        Quote saved = quoteRepository.save(quote);
        return new ResponseEntity<Quote>(saved, HttpStatus.CREATED);
    } catch (Exception e) {
        System.out.println(e.getMessage());
        return new ResponseEntity<Quote>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

@PutMapping("/quotes/{id}")
public ResponseEntity<Quote> updateQuote(@PathVariable("id") Integer id, @RequestBody
Quote quote) {
    try {
        Optional<Quote> existingQuote = quoteRepository.findById(id);

        if(existingQuote.isPresent()){
            Quote updatedQuote = existingQuote.get();
            updatedQuote.setAuthor(quote.getAuthor());
            updatedQuote.setQuote(quote.getQuote());

            return new ResponseEntity<Quote>(updatedQuote, HttpStatus.OK);
        } else {
            return new ResponseEntity<Quote>(HttpStatus.NOT_FOUND);
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
        return new ResponseEntity<Quote>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

>DeleteMapping("/quotes/{id}")
public ResponseEntity<HttpStatus> deleteQuote(@PathVariable("id") Integer id) {
    Optional<Quote> quote = quoteRepository.findById(id);
    if (quote.isPresent()) {
        quoteRepository.deleteById(id);
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
    } else {
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
}

```

新增資料庫設定

application.yaml

為服務存取的後端資料庫新增設定。編輯 (或建立，如果不存在) `src/main/resources` 下名為 `application.yaml` 的檔案，並為後端新增參數化 Spring 設定。

```
target: local

spring:
  config:
    activate:
      on-profile: cloud-dev
  datasource:
    url: 'jdbc:postgresql://${DB_HOST:127.0.0.1}/${DB_NAME:quote_db}'
    username: '${DB_USER:user}'
    password: '${DB_PASS:password}'
  jpa:
    properties:
      hibernate:
        jdbc:
          lob:
            non_contextual_creation: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
    hibernate:
      ddl-auto: update
```

新增資料庫遷移

在 `src/main/resources` 下建立資料夾 `db/migration`

建立 SQL 檔案： `V1__create_quotes_table.sql`

將下列內容貼到檔案中

V1__create_quotes_table.sql

```
CREATE TABLE quotes(
  id INTEGER PRIMARY KEY,
  quote VARCHAR(1024),
  author VARCHAR(256)
);

INSERT INTO quotes (id,quote,author) VALUES (1,'Never, never, never give up','Winston Churchill');
INSERT INTO quotes (id,quote,author) VALUES (2,'While there's life, there's hope','Marcus Tullius Cicero');
INSERT INTO quotes (id,quote,author) VALUES (3,'Failure is success in progress','Anonymous');
INSERT INTO quotes (id,quote,author) VALUES (4,'Success demands singleness of purpose','Vincent Lombardi');
INSERT INTO quotes (id,quote,author) VALUES (5,'The shortest answer is doing','Lord Herbert');
```

Kubernetes 設定

在 `deployment.yaml` 檔案中加入下列內容，即可讓應用程式連線至 Cloud SQL 執行個體。

- TARGET - 設定變數，指出應用程式執行的環境
- SPRING_PROFILES_ACTIVE - 顯示作用中的 Spring 設定檔，該設定檔會設為 `cloud-dev`

- DB_HOST - 資料庫的私人 IP，建立資料庫執行個體時已記錄，或點選 Google Cloud 控制台導覽選單中的 `SQL` 即可查看 - 請變更值！
- DB_USER 和 DB_PASS - 如同在 CloudSQL 執行個體設定中設定，並以密鑰形式儲存在 GCP 中

使用下列內容更新 deployment.yaml。

deployment.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: demo-app
  labels:
    app: demo-app
spec:
  ports:
    - port: 8080
      protocol: TCP
  clusterIP: None
  selector:
    app: demo-app
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
  labels:
    app: demo-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
        - name: demo-app
          image: demo-app
          env:
            - name: PORT
              value: "8080"
            - name: TARGET
              value: "Local Dev - CloudSQL Database - K8s Cluster"
            - name: SPRING_PROFILES_ACTIVE
              value: cloud-dev
            - name: DB_HOST
              value: ${DB_INSTANCE_IP}
            - name: DB_PORT
              value: "5432"
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: gke-cloud-sql-secrets
                  key: username
            - name: DB_PASS
              valueFrom:
                secretKeyRef:

```

```

      name: gke-cloud-sql-secrets
      key: password
- name: DB_NAME
  valueFrom:
    secretKeyRef:
      name: gke-cloud-sql-secrets
      key: database

```

在終端機中執行下列指令，將 DB_HOST 值替換為資料庫的位址：

```

export DB_INSTANCE_IP=$(gcloud sql instances describe quote-db-instance \
  --format=json | jq \
  --raw-output ".ipAddresses[].ipAddress")

envsubst < deployment.yaml > deployment.new && mv deployment.new deployment.yaml

```

開啟 deployment.yaml，確認 DB_HOST 值已更新為執行個體 IP。

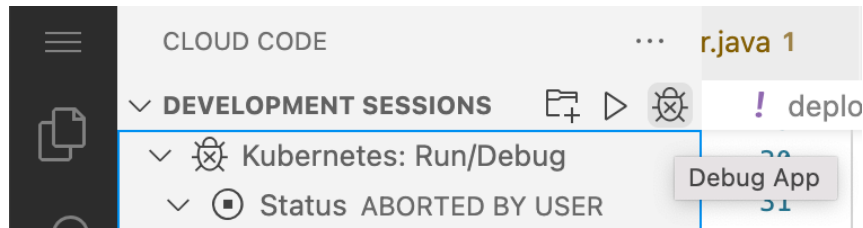
```

! deployment.yaml > {} spec > {} template > {} spec > [ ] containers > [ ] env
30  spec:
31    containers:
32  - name: demo-app
33    image: demo-app
34    env:
35  - name: PORT
36    value: "8080"
37  - name: TARGET
38    value: "Local Dev - CloudSQL Database - K8s Cluster"
39  - name: SPRING_PROFILES_ACTIVE
40    value: cloud-dev
41  - name: DB_HOST
42    value: 10.55.16.3
43  - name: DB_PORT
44    value: "5432"
45  - name: DB_USER
46    valueFrom:
47    secretKeyRef:
      Created 39 minutes ago
      name: gke-cloud-sql-secrets
      key: username

```

部署及驗證應用程式

1. 在 Cloud Shell 編輯器底部的窗格中，選取「Cloud Code」，然後選取畫面頂端的「Debug on Kubernetes」（在 Kubernetes 上偵錯）。



2. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/demo-app status completed successfully`，並列出網址：「Forwarded URL from service demo-app: <http://localhost:8080>」。請注意，有時通訊埠可能不同，例如 8081。如果是，請設定適當的值。在終端機中設定網址的值

```
export URL=localhost:8080
```

3. 查看隨機名言

在終端機中，對 `random-quote` 端點多次執行下列指令。觀察重複呼叫傳回不同報價

```
curl $URL/random-quote | jq
```

4. 新增報價

使用下列指令建立 ID 為 6 的新報價，並觀察要求是否回傳

```
curl -H 'Content-Type: application/json' -d '{"id":"6","author":"Henry David Thoreau","quote":"Go confidently in the direction of your dreams! Live the life you have imagined"}' -X POST $URL/quotes
```

5. 刪除報價

現在請使用刪除方法刪除剛才新增的引言，並觀察 `HTTP/1.1 204` 回應代碼。

```
curl -v -X DELETE $URL/quotes/6
```

6. 伺服器錯誤

在項目已刪除後再次執行最後的要求，就會發生錯誤狀態

```
curl -v -X DELETE $URL/quotes/6
```

請注意，回應會傳回 `HTTP:500 Internal Server Error`。

對應用程式進行偵錯

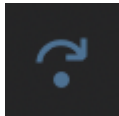
在上一節中，您嘗試刪除資料庫中沒有的項目時，應用程式發生錯誤。在本節中，您將設定中斷點來找出問題。錯誤發生在 `DELETE` 作業中，因此您會使用 `QuoteController` 類別。

1. 開啟「`src/main/java/com/example/springboot/QuoteController.java`」
2. 找出 `deleteQuote()` 方法
3. 找到以下這行程式碼：`Optional<Quote> quote = quoteRepository.findById(id);`
4. 點選行號左側的空白處，在該行設定中斷點。

5. 系統會顯示紅色指標，表示已設定中斷點
6. 再次執行 `delete` 指令

```
curl -v -X DELETE $URL/quotes/6
```

7. 按一下左欄中的圖示，切換回偵錯檢視畫面
8. 觀察在 `QuoteController` 類別中停止的偵錯行。
9. 在偵錯工具中，按一下 `step over` 圖示



10. 請注意，程式碼會向用戶端傳回「內部伺服器錯誤」HTTP 500，這並非理想做法。

```
Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080 (#0)
> DELETE /quotes/6 HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 500
< Content-Length: 0
< Date:
<
* Connection #0 to host 127.0.0.1 left intact
```

更新程式碼

程式碼不正確，應重構 `else` 區塊，傳回 HTTP 404 找不到狀態碼。

修正錯誤。

1. 在偵錯工作階段仍在執行的情況下，按下偵錯控制面板中的「繼續」按鈕，完成要求。
2. 接著，將 `else` 區塊變更為下列程式碼：

```
else {
    return new ResponseEntity<HttpStatus>(HttpStatus.NOT_FOUND);
}
```

方法應如下所示：

```
@DeleteMapping("/quotes/{id}")
public ResponseEntity<HttpStatus> deleteQuote(@PathVariable("id") Integer id) {
    Optional<Quote> quote = quoteRepository.findById(id);
    if (quote.isPresent()) {
        quoteRepository.deleteById(id);
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
    } else {
        return new ResponseEntity<HttpStatus>(HttpStatus.NOT_FOUND);
    }
}
```

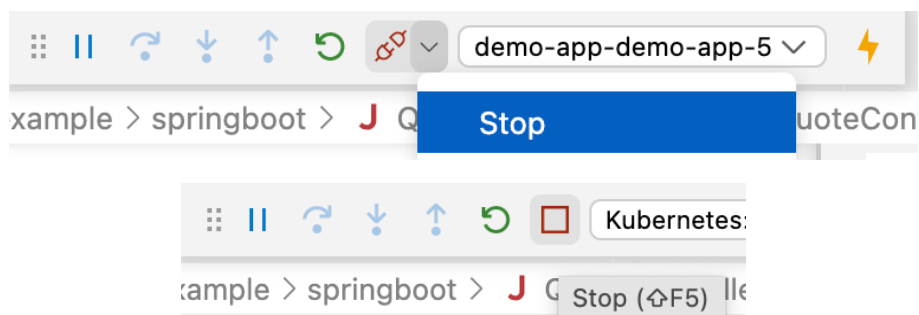
3. 重新執行刪除指令

```
curl -v -X DELETE $URL/quotes/6
```

4. 逐步執行偵錯工具，觀察傳回給呼叫端的 HTTP 404 Not Found。

```
Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080 (#0)
> DELETE /quotes/6 HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404
< Content-Length: 0
< Date:
<
* Connection #0 to host 127.0.0.1 left intact
```

5. 按一下偵錯工具列中的紅色正方形，停止偵錯工作階段



6. 恭喜

恭喜！在本實驗室中，您從頭開始建立新的 Java 應用程式，並設定該應用程式，使其能有效與容器搭配運作。然後，您按照傳統應用程式堆疊中的相同開發人員流程，將應用程式部署至遠端 GKE 叢集並進行偵錯。

您學到的內容

- 使用 Cloud Workstations 進行 InnerLoop 開發
 - 建立新的 Java 啟動應用程式
 - 逐步完成開發程序
 - 開發簡易的 CRUD REST 服務
 - 對 GKE 叢集上的應用程式進行偵錯
 - 將應用程式連線至 Cloud SQL 資料庫
-